Lab 5: Static Code Analysis

PES1UG24CS297 | Nehaa Joshi

1. Objective and Introduction

The objective of this lab was to improve the quality, readability and security of a Python program using three static code analysis tools: Pylint, Bandit and Flake8. These tools help identify potential problems in code without requiring the program to be executed.

Pylint was used to check code quality, naming conventions and programming practices. Bandit was used to identify security issues, while Flake8 checked formatting and compliance with Python style guidelines.

For this lab, I analysed the parking lot management program, identified the issues reported by the tools, and modified the code to fix them. After making the changes, I ran the tools again to verify the results and checked that the program still executed correctly.

2. Issues Identified and Fixes Applied

Tool	Issue	Original line(s)	Description	Fix applied
Pylint	Missing module docstring	1	The module did not explain its overall purpose.	Added a module-level docstring.
Pylint	Missing function docstrings	15, 27, 36, 46, 52, 56, 60, 75	Several functions did not explain what they do.	Added docstrings describing the functions.
Pylint	Mutable default argument	15	Using [] as a default argument can cause a list to be shared between function calls.	Changed the default to None and initialized the list inside the function.
Pylint	Naming conventions	12, 15, 27, 36, 46, 52, 56, 60, 67	Some names did not follow Python naming conventions.	Used uppercase for constants and snake_case for functions and variables.

Pylint	Global statement	16	The function modified a global variable directly.	Changed the collection total to a mutable dictionary and updated its value without a global statement.
Pylint	Long line and string formatting	23	A line exceeded the recommended length, and a regular string could be formatted as an f-string.	Split the long line and used an f-string where appropriate.
Pylint	Bare exception	32	A bare except can hide unexpected errors.	Removed the broad exception pattern and handled invalid expressions explicitly.
Pylint	Comparison with None	38	The code used == None instead of the recommended identity check.	Replaced it with is None.
Pylint	Unused variables	37, 47, 69	Some assigned variables were not used.	Removed unused variables and simplified the related code.
Pylint	Unsafe eval()	48	eval() can execute arbitrary Python expressions.	Replaced it with a restricted arithmetic evaluator using the AST module.
Pylint	File handling	61	The file was opened without an explicit encoding or a context manager.	Used Path.write_text() with UTF-8 encoding.
Pylint	Unused import	4	The math module was imported but not used.	Removed the unused import.
Bandit	try/except/pass pattern	32	Silently ignoring exceptions can hide errors.	Removed the flagged pattern and handled invalid expressions explicitly.
Bandit	Unsafe eval()	48	Arbitrary expression evaluation can introduce security risks.	Used AST validation and a restricted arithmetic evaluator.

Bandit	Weak MD5 hashing	53	MD5 is not suitable for security-sensitive hashing.	Replaced MD5 with SHA-256.
Bandit	Insecure random PIN generation	57	The random module is not intended for security-sensitive random values.	Used the secrets module to generate the ticket PIN.
Bandit	Shell command execution	64	Using os.system() can expose the program to command-injection risks.	Removed shell execution and used Python file-handling functionality.
Flake8	Unused import	4	The math import was not used.	Removed the unused import.
Flake8	Long lines	7, 8, 9, 23	Four lines exceeded the configured line-length limit.	Reformatted the lines for readability.
Flake8	Bare except	32	The code used a bare exception handler.	Removed the broad exception pattern.
Flake8	Unused variable overtime_fee	37	The variable was assigned but never used.	Removed the unused variable.
Flake8	Comparison with None	38	The code compared a value to None using ==.	Changed the comparison to is None.
Flake8	Unused variable base_rate	47	The variable was assigned but never used in the original code.	Removed the unused assignment and implemented the fee calculation safely.
Flake8	Missing final newline	88	The file did not end with a newline.	Added the final newline.

3. Before-and-After Results

After making the changes, I ran all three tools again and compared their reports with the original results.

Tool	Before fixing	After fixing	Result
Pylint	4.10/10	10.00/10	Improved code quality score
Bandit	5 findings: 2 high, 1 medium and 2 low	0 findings	Security findings resolved
Flake8	10 reported violations	0 violations	Formatting issues resolved

The Pylint score increased from 4.10 to 10.00 after the code-quality improvements. Bandit no longer reported any security findings, and Flake8 produced no output after the formatting fixes.

I also executed the updated program to check that its main functionality continued to work. It calculated the parking fee, generated the vehicle hash and ticket PIN, exported the parking manifest, and displayed the vacant-slot count.

4. Reflection Questions and Answers

1. What is the importance of static code analysis?

Static code analysis helps identify potential errors, security vulnerabilities and style violations before the software is deployed. It makes it easier to detect problems early and improves the readability, maintainability and reliability of the code.

2. What is the difference between Pylint, Bandit and Flake8?

Pylint focuses on overall code quality, including naming conventions, unused variables, documentation and programming practices. Bandit focuses on security risks and potentially unsafe coding patterns. Flake8 checks Python style guidelines, formatting, whitespace and related errors. Using all three provides a more complete analysis than relying on only one tool.

3. What were the major issues found in the parking lot program?

Some of the major issues included the use of `eval()` for fee calculation, MD5 hashing, insecure random PIN generation, shell command execution, mutable default arguments, unused variables and formatting violations. These issues affected the security, maintainability and readability of the program.

4. Which fix was the most important, and why?

Replacing `eval()` was one of the most important fixes because unrestricted evaluation can execute arbitrary Python code. I replaced it with an evaluator that accepts only a limited set of arithmetic operations. I also replaced MD5 with SHA-256 and used the `secrets` module for ticket PIN generation to improve security.

5. What did you learn from using the three tools?

I learned that different static analysis tools identify different types of problems. Pylint helped improve the structure and quality of the code, Bandit helped identify security risks, and Flake8 helped maintain consistent formatting. I also learned that resolving warnings and rerunning the tools is an effective way to verify improvements.

6. What would you do differently in future projects?

In future projects, I would run static analysis tools regularly during development rather than waiting until the end. I would also pay more attention to security while implementing features and include static analysis checks in the development workflow to identify issues earlier.

5. Conclusion

This lab helped me understand how static code analysis can improve the quality and security of a Python program. By using Pylint, Bandit and Flake8, I identified and fixed issues related to code structure, security and formatting in the parking lot management system.

After the changes, the Pylint score increased to 10.00/10, Bandit reported no issues, and Flake8 reported no formatting violations. The program also continued to execute successfully. Overall, the lab showed me the importance of checking code quality and security throughout the software development process, rather than focusing only on whether the program runs.